



Cyberscope

A *TAC Security* Company

Audit Report

Hydranet

July 2026

Repository <https://github.com/Offchain-DEX/lithium-contracts>

Commit [1e0ec8c651f66ca065ee1d4fb185915f1f79f2cd](#)

Audited by © cyberscope

Table of Contents

Table of Contents	1
Risk Classification	3
Review	4
Audit Updates	4
Source Files	4
Overview	6
Channel State and Pool Model	6
Cooperative Channel Updates	6
Unilateral Settlement and Disputes	7
Finalizing Unilateral Settlement	7
Revocation and Fraud Penalties	8
Pool Deposits and Withdrawals	8
Batch Execution	9
Findings Breakdown	10
Diagnostics	11
AME - Address Manipulation Exploit	12
Description	12
Recommendation	12
Team Update	13
CREF - Cumulative Revocation Encoding Fragility	14
Description	14
Recommendation	14
Team Update	15
DDO - Dispute Data Overwritten	16
Description	16
Recommendation	16
Team Update	17
HRDM - HTLC Resolution Deadline Mismatch	18
Description	18
Recommendation	19
Team Update	20
MMDP - Missing Minimum Dispute Period	21
Description	21
Recommendation	21
Team Update	21
NPSD - NFT Permanent Settlement Denial	22
Description	22
Recommendation	22
Team Update	23

NVB - Nonce-Based Validation Bypass	24
Description	24
Recommendation	24
Team Update	25
PDS - Permissionless Dispute Settlement	26
Description	26
Recommendation	26
Team Update	26
PTAI - Potential Transfer Amount Inconsistency	27
Description	27
Recommendation	28
Team Update	28
SSS - Stale State Selection	29
Description	29
Recommendation	29
Team Update	30
UPH - Unsafe Packed Hashing	31
Description	31
Recommendation	32
Team Update	32
Function Analysis	33
Summary	37
Disclaimer	38
About Cyberscope	39

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Repository	https://github.com/Offchain-DEX/lithium-contracts
Commit	1e0ec8c651f66ca065ee1d4fb185915f1f79f2cd

Audit Updates

Initial Audit	08 Dec 2025 https://github.com/cyberscope-io/audits/blob/main/2-hdn/v1/audit.pdf
Corrected Phase 2	03 Mar 2026 https://github.com/cyberscope-io/audits/blob/main/2-hdn/v2/audit.pdf
Corrected Phase 3	10 Apr 2026 https://github.com/cyberscope-io/audits/blob/main/2-hdn/v3/audit.pdf
Corrected Phase 4	18 Jul 2026

Source Files

Filename	SHA256
Lithium.sol	b40d9a3730f9e6459c32095b2d7bbe9218 556b053986f860fe876e2205b28204
libraries/LithiumVerification.sol	8bed5ccc881473551914d9d9a5f0dd36a6 0fa79f291eb5dd8f21dc6ec7b82a4d
libraries/LithiumUpdater.sol	91af8ec23c226c79d35740c3dbf70b5a379 a9706a0f08a79af26ea47036d6790

libraries/LithiumTransfer.sol	d989e6a4f8ca6832e2e2eb3140ddc7bede 5472029ddd1782b8525727a76fb216
libraries/LithiumSettler.sol	39edcd96a9751c958cbea417fe35521dee af09262ada58c4c2168c298a32d067
libraries/LithiumRevoker.sol	39956662d5fa461ba5af890fb3a1c7a0343 3f798dfe8fad4ca61ab83f334eb54
interfaces/ILithium.sol	4b9271be8d8fd29bcdeb41335dcdc788fa b63a925f41d847c38e1146739d6b28

Overview

Lithium is designed as a two-party, multi-asset payment channel system combined with a per-user custody pool. Each payment channel is uniquely identified by the tuple (alice, bob, channelNumber), representing the two participants and a logical channel instance between them. Within a channel, multiple assets can coexist, and each asset is further identified by an assetVariant, an assetAddress, and a tokenNumberId. This design allows a single channel to simultaneously manage native assets, ERC20-like tokens, or NFT-style assets without interference.

The contract maintains three core mappings to track channel and user state. The first mapping, `assetChannels`, stores a commitment hash for each asset-specific channel, representing the current agreed-upon state between the parties. The second mapping, `assetDisputes`, stores an active dispute object whenever a unilateral settlement process has been initiated. The third mapping, `claimable`, stores escrowed funds assigned to users during adversarial settlement or revocation, which can be withdrawn later via a pull-payment mechanism. In parallel to channels, Lithium also maintains a per-user custody pool. This pool is not shared liquidity; instead, it is a personal ledger where each user has balances per asset key. Users may deposit assets into the contract, have them credited to their pool balance, and later withdraw them independently of channel activity.

Channel State and Pool Model

A channel in Lithium represents an off-chain agreement whose latest valid state is summarized on-chain as a hash commitment. These commitments encode the channel nonce, the total asset balance, and which party authored the update. The pool mechanism acts as a custody layer that backs channel balances. Deposits increase a user's pool balance, withdrawals decrease it, and channel updates may move value between a user's pool and the channel state. Importantly, the pool does not automatically intermingle funds between users; each participant's balances are isolated and tracked independently.

Cooperative Channel Updates

The `updateChannel` function enables cooperative updates to a channel state when both Alice and Bob agree. This function forwards execution to the `LithiumUpdater` logic, which ensures that the caller is one of the channel participants and verifies both

signatures. For each asset included in the update, the updater processes the transition via an internal asset-specific routine.

During an asset update, the contract compares the total balance of the previous state with that of the new state. If the sum is unchanged, the function returns early without modifying storage, meaning no nonce progression or commitment updates occur. When the total balance changes, the updater validates that the previously stored commitment matches the expected nonce and balances, and processes deposits or withdrawals against the users' pool balances. Once validated, the new state is committed on-chain by storing a hash derived from the new nonce, updated balance sum, and the message sender. The wrapper function ensures that the correct amount of ETH is supplied for native asset updates.

Unilateral Settlement and Disputes

The `unilateralSettleChannel` function allows either party to bring the latest signed channel state on-chain to initiate or advance a dispute. For each asset, signatures are strictly verified, and the settlement logic processes the asset-specific dispute. If no dispute exists, only Alice or Bob may initiate it. Once a dispute is active, subsequent calls are not restricted by sender identity, enabling watchtower-style monitoring where any third party holding a valid co-signed state can submit updates on behalf of an offline participant.

The contract enforces monotonic nonce progression by requiring that submitted nonces are greater than or equal to the stored dispute nonce. When hashlocked payments are involved, the contract reconstructs the Merkle root from the provided hashlock data and validates any revealed preimages against it. If the signed state commits to a non-zero Merkle root, the full payment list must be provided; empty submissions against a non-zero root are rejected. Higher-nonce overrides replace the dispute state entirely and reset the dispute deadline, while same-nonce updates only allow incremental preimage reveals without extending the dispute window.

Finalizing Unilateral Settlement

Once the dispute period has elapsed, `confirmUnilateralSettlement` can be called to finalize the channel. The non-disputer counterparty may confirm immediately at any time without waiting for the deadline. For each disputed asset, the contract checks that a dispute exists and applies the timing rule. The base payouts are computed from the disputed balances and then adjusted according to any revealed hashlock preimages,

effectively netting conditional payments. Fraud penalties are not applied during settlement confirmation; penalties are handled exclusively by the revocation mechanism. Assets are distributed using a deterministic anti-griefing pattern: the caller's own allocation is pushed directly, while the counterparty's allocation is moved into the `claimable` escrow for later pull-withdrawal via `claimAsset`. This ensures settlement remains finalizable regardless of the counterparty's receive behavior. Both the channel commitment and dispute state are cleared from storage, with the commitment slot set to a sentinel value that permanently prevents the channel from being reopened with the same parameters.

Revocation and Fraud Penalties

The `penalizeRevokedChannel` function enables any party to penalize a participant who broadcasted a revoked channel state. Each revocation is signed unilaterally by the revoking party, and the contract recovers the signer via ECDSA to identify which participant committed the revocation. Two enforcement paths exist: the dispute path handles cumulative revocations (covering all states up to a given nonce) against an active unilateral settlement, while the cooperative path handles nonce-specific revocations against a stale cooperative update stored in the channel commitment. All funds from the penalized channel are assigned to the non-revoking party (the victim), making the function permissionless and front-running unprofitable since the reward recipient is deterministic. The victim receives funds either directly (if they are the caller) or via the `claimable` escrow. After execution, the channel and any associated dispute are cleared, with the commitment slot set to the sentinel value.

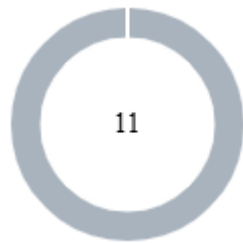
Pool Deposits and Withdrawals

The pool functions manage user custody balances. Deposits credit the caller's pool balance for a given asset key. For native assets, the required ETH amount must be sent with the transaction, while token deposits transfer assets from the user to the contract. Withdrawals verify that the user has sufficient credited balance, decrement their pool entry, and transfer the asset back to the user. Additionally, the contract's receive function treats any direct ETH transfer as a native asset deposit credited to the sender. The `claimAsset` function allows users to withdraw escrowed funds that were assigned to them during adversarial settlement or revocation.

Batch Execution

The `batchExecute` function allows multiple operations to be bundled into a single transaction. Each operation is decoded and executed in sequence using the corresponding internal logic. Supported operations include channel updates, unilateral settlements, settlement confirmations, revocation penalties, pool deposits, pool withdrawals, and asset claims. ETH requirements for updates and deposits are aggregated and validated at the end of execution.

Findings Breakdown



● Critical	0
● Medium	0
● Minor / Informative	11

Severity	Unresolved	Acknowledged	Resolved	Other
● Critical	0	0	0	0
● Medium	0	0	0	0
● Minor / Informative	0	11	0	0

Diagnostics

● Critical
 ● Medium
 ● Minor / Informative

Severity	Code	Description	Status
●	AME	Address Manipulation Exploit	Acknowledged
●	CREF	Cumulative Revocation Encoding Fragility	Acknowledged
●	DDO	Dispute Data Overwritten	Acknowledged
●	HRDM	HTLC Resolution Deadline Mismatch	Acknowledged
●	MMDP	Missing Minimum Dispute Period	Acknowledged
●	NPSD	NFT Permanent Settlement Denial	Acknowledged
●	NVB	Nonce-Based Validation Bypass	Acknowledged
●	PDS	Permissionless Dispute Settlement	Acknowledged
●	PTAI	Potential Transfer Amount Inconsistency	Acknowledged
●	SSS	Stale State Selection	Acknowledged
●	UPH	Unsafe Packed Hashing	Acknowledged

AME - Address Manipulation Exploit

Criticality	Minor / Informative
Location	Lithium.sol
Status	Acknowledged

Description

The contract's design includes functions that accept external contract addresses as parameters without performing adequate validation or authenticity checks. This lack of verification introduces a significant security risk, as input addresses could be controlled by attackers and point to malicious contracts. Such vulnerabilities could enable attackers to exploit these functions, potentially leading to unauthorized actions or the execution of malicious code under the guise of legitimate operations.

```
struct AssetState {  
    ...  
    address assetAddress;  
    ...  
}  
  
struct Revocation {  
    ...  
    address assetAddress;  
    ...  
}
```

Recommendation

To mitigate this risk and enhance the contract's security posture, it is imperative to incorporate comprehensive validation mechanisms for any external contract addresses passed as parameters to functions. This could include checks against a whitelist of approved addresses, verification that the address implements a specific contract interface or other methods that confirm the legitimacy and integrity of the external contract. Implementing such validations helps prevent malicious exploits and ensures that only trusted contracts can interact with sensitive functions.

Team Update

The team has acknowledged that this is not a security issue and states:

This is not a vulnerability; it is an intentional design feature. The protocol is permissionless by design and must accept arbitrary asset addresses to fulfill its core value proposition. The security model does not rely on asset whitelisting but instead on isolation, mutual agreement, and reentrancy protection.

CREF - Cumulative Revocation Encoding Fragility

Criticality	Minor / Informative
Location	libraries/LithiumRevoker.sol#L169
Status	Acknowledged

Description

The `Revocation` struct overloads the `balanceSum` field with two distinct meanings: a value of zero indicates a cumulative revocation covering all states up to and including `nonce`, while a non-zero value indicates a nonce-specific revocation matching the exact `(nonce, balanceSum, revoker)` tuple in the cooperative storage. An off-chain implementation that simply reads the actual balance sum from a state and signs it can accidentally produce a "vacuous" revocation when the state's actual balance sum happens to be zero. The contract interprets this as a cumulative revocation covering every prior nonce, including states with substantial funds, allowing the counterparty to penalize the signer for far more than they intended.

```
``solidity
struct Revocation {
    AssetVariant assetVariant;
    address assetAddress;
    uint256 tokenId;
    uint256 nonce;
    uint256 balanceSum;
}
```

Recommendation

The team is advised to add an explicit `bool cumulative` field to the `Revocation` struct and include it in the signed message hash. This will eliminate the encoding ambiguity and force the off-chain layer to make an explicit choice between cumulative and nonce-specific.

Team Update

The team has acknowledged that this is not a security issue and states:

No code change is needed. The `balanceSum == 0` conflation is safe because zero-balance states have no economic value. The design is fully documented in NatSpec and `PROTOCOL-FLOWS.md`. Adding a bool cumulative field would increase signature size and gas cost without any security improvement.

DDO - Dispute Data Overwritten

Criticality	Minor / Informative
Location	libraries/LithiumSettler.sol#L169,268
Status	Acknowledged

Description

The contract fully overwrites the stored hashlockPayments array whenever a dispute update is processed with deadline == 0 or a higher nonce, replacing all previously stored payments and discarding any revealed preimages. However, during an active dispute, preimage revelation is meant to be a one-way, irreversible action that progressively resolves conditional payments. By allowing a later state update to erase this data, the protocol enables a party to nullify the counterparty's progress simply by submitting a higher-nonce state that changes the Merkle root, even if the earlier state was already partially resolved on-chain.

```
if (paymentsLength > 0) {
    dispute.hashlockPayments = new HashlockPayment[] (paymentsLength);
    for (uint256 j = 0; j < paymentsLength;) {
        dispute.hashlockPayments[j] = hashlockPayments[j];
        unchecked {
            ++j;
        }
    }
} else if (dispute.hashlockPayments.length > 0) {
    delete dispute.hashlockPayments;
}
```

Recommendation

The team is advised to clearly define whether higher-nonce states are allowed to supersede an active dispute and to enforce rules that preserve dispute fairness. A safer design is to freeze the disputed state at the nonce that opened the dispute, allowing only additive actions such as preimage revelation until settlement. If higher-nonce challenges during a dispute are intentionally supported, the contract must ensure that previously revealed

preimages are not invalidated or erased and that any new state is strictly compatible with already-resolved conditions, preventing one party from rolling back on-chain progress.

Team Update

The team has acknowledged that this is not a security issue and states:

Intentional design. The overwriting of dispute state upon broadcasting a co-signed state with a higher nonce is necessary and intended by the protocol's security mechanisms. Since nonces are monotonically increasing, a co-signed state with a higher nonce represents a more recent channel state that must take precedence to reflect the latest agreed-upon conditions between parties.

HRDM - HTLC Resolution Deadline Mismatch

Criticality	Minor / Informative
Location	libraries/LithiumVerification.sol#L260,291 libraries/LithiumSettler.sol#L535,550,603
Status	Acknowledged

Description

Each `HashlockPayment` has its own `resolutionDeadline`, which is independent of the channel's `disputePeriod` and resulting `disputeDeadline`. The preimage validation in `calculateMerkleRootAndVerifyPreimages()` and `calculateMerkleRootAndVerifyPreimagesCalldata()` reverts when `block.timestamp > payment.resolutionDeadline`, meaning an HTLC can become unresolvable on-chain even while the channel dispute remains active:

```
if (payment.preimage != bytes32(0)) {
    if (sha256(abi.encodePacked(payment.preimage)) != payment.hash)
    revert InvalidPreimage();
    if (block.timestamp > payment.resolutionDeadline) revert
    PaymentDeadlinePassed();
}
```

Conversely, the channel can become finalizable while an unresolved HTLC is still within its valid resolution period. Once the dispute deadline is reached, any caller may confirm the settlement. Additionally, the non-disputing participant is permitted to confirm before the dispute deadline:

```
bool disputerIsAlice = dispute.disputerIsAlice;
if (!(disputerIsAlice ? msg.sender == bob : msg.sender == alice)) {
    if (block.timestamp < dispute.disputeDeadline) revert
    DisputePeriodNotPassed();
}
```

During confirmation, `_resolveHTLCs()` applies only payments whose preimages have already been recorded:

```
if (payment.preimage != bytes32(0)) {  
    if (payment.direction == PaymentDirection.BobToAlice) bobSent +=  
    payment.amount;  
    else aliceSent += payment.amount;  
}
```

An unresolved payment is ignored irrespective of whether its `resolutionDeadline` remains active. The contract then permanently closes the channel and deletes the dispute data, including all stored hashlock payments:

```
store.assetChannels[assetChannelId] = CLOSED_CHANNEL_SENTINEL;  
delete store.assetDisputes[assetChannelId];
```

The independence of these deadlines therefore creates two inconsistent timing conditions: an HTLC may expire while the channel dispute remains active, or the channel may be finalized while an HTLC remains valid. In the latter case, finalization permanently prevents the recipient from revealing the preimage and claiming the conditional payment before its signed resolution deadline.

Recommendation

The team is advised to resolve HTLCs independently from channel finalization so that differences between the HTLC resolution deadline and the channel dispute deadline cannot invalidate a live conditional payment or delay settlement unnecessarily. One possible approach would be to transfer unresolved HTLC amounts into dedicated conditional escrow during channel confirmation, allowing recipients to claim them with valid preimages until their respective resolution deadlines and allowing payers to reclaim them after expiry. This preserves the intended HTLC lifecycle regardless of whether an HTLC expires before the dispute ends or remains active when the channel is finalized.

Team Update

The team has acknowledged that this is not a security issue and states:

No code change is needed. The `resolutionDeadline` and `disputePeriod` are correctly independent: the former is the receiver's deadline to reveal a preimage (before which they can always act, including atomically within a dispute transaction), while the latter governs challenge windows for state disputes. On-chain enforcement would reject legitimate HTLCs in multi-hop routing. This matches the Lightning Network model.

MMDP - Missing Minimum Dispute Period

Criticality	Minor / Informative
Location	libraries/LithiumSettler.sol#L376
Status	Acknowledged

Description

The contract validates that `block.timestamp + disputePeriod` does not overflow `uint64` but does not enforce any lower bound on `disputePeriod`. If a counterparty signs a state with `disputePeriod = 0` (or any value small enough that `block.timestamp == disputeDeadline`), an attacker can atomically open a unilateral settlement and immediately confirm it via `batchExecute`, giving the counterparty zero time to challenge with a newer state or revocation. This completely defeats the dispute mechanism and bypasses the entire revocation defense, since revocations also require an active dispute window for the cumulative path.

```
uint256 deadline = block.timestamp + uint256(disputePeriod);
if (deadline > type(uint64).max) revert InvalidDisputePeriod();
dispute.disputeDeadline = uint64(deadline);
```

Recommendation

The team is advised to add an on-chain lower bound on `disputePeriod`. This will eliminate the atomic state-state settlement attack vector entirely without requiring trust in off-chain signing logic.

Team Update

The team has acknowledged that this is not a security issue and states:

No smart contract change is needed. The disputePeriod is co-signed by both parties, the off-chain requirement is documented in NatSpec (ILithium.sol, LithiumSettler.sol), README.md, and PROTOCOL-FLOWS.md, and on-chain enforcement would be arbitrary. This follows the same pattern as Lightning Network's to_self_delay negotiation.

NPSD - NFT Permanent Settlement Denial

Criticality	Minor / Informative
Location	libraries/LithiumSettler.sol#L116,215 libraries/LithiumTransfer.sol#L126 Lithium.sol#L256
Status	Acknowledged

Description

The unilateral settlement entry point does not validate that ERC721 balances or hashlock payment amounts are restricted to zero or one during dispute creation. The strict `amount != 1` check in `withdrawAssetTo` only fires at the final transfer step. If a state is co-signed with an ERC721 balance greater than one, or if HTLC netting produces a non-unit ERC721 balance, the confirmation transaction reverts permanently and the NFT becomes locked in the contract with no recovery path.

```
function _unilateralSettleAssetChannel(  
    Storage storage store,  
    ChannelConfig memory channelConfig,  
    AssetState memory assetState,  
    HashlockPayment[] memory hashlockPayments  
) internal  
  
...  
} else if (variant == AssetVariant.ERC721) {  
    if (amount != 1) revert InvalidERC721Amount();  
    IERC721(asset).safeTransferFrom(address(this), to, id);  
}
```

Recommendation

The team is advised to implement strict validation checks within the unilateral settlement entry point to ensure that all balances and hashlock payment amounts for ERC721 assets are restricted to values of zero or one. This will prevent the creation of invalid dispute states that can never be finalized and ensure that the on-chain netting logic remains compatible with the non-fungible nature of the tokens.

Team Update

The team has acknowledged that this is not a security issue and states:

- 1. Both parties must co-sign the state. An NFT balance > 1 requires both Alice and Bob to explicitly sign an invalid state. This is not a unilateral attack.*
- 2. The transfer layer enforces correctness. `LithiumTransfer.withdrawAssetTo` reverts with `InvalidERC721Amount()` if `amount != 1`. This is a safety check.*
- 3. Off-chain enforcement. The off-chain node validates ERC721 balances are restricted to 0 or 1 before co-signing any state. An invalid state can only arise from a bug in or deliberate manipulation of the off-chain software.*
- 4. No silent fund loss. If an invalid state is co-signed and disputed, the confirmation reverts — the NFT remains in the contract. This is preferable to distributing an invalid amount.*

NVB - Nonce-Based Validation Bypass

Criticality	Minor / Informative
Location	libraries/LithiumSettler.sol#L353
Status	Acknowledged

Description

The contract permits validating the “previous on-chain state” by comparing the stored commitment against a hash constructed using the submitted `nonce` instead of the actual stored `onchainNonce`. However, this relaxes the core binding between the chain’s recorded state and the state being settled, effectively allowing a caller to claim that the stored commitment corresponds to whatever nonce they are attempting to use, rather than the nonce that is actually recorded on-chain.

```
if (
    storedStateHash != keccak256(abi.encodePacked(onchainNonce,
    balanceSum, alice))
    && storedStateHash != keccak256(abi.encodePacked(onchainNonce,
    balanceSum, bob))
    && storedStateHash != keccak256(abi.encodePacked(nonce,
    balanceSum, alice))
    && storedStateHash != keccak256(abi.encodePacked(nonce,
    balanceSum, bob))
) {
    revert InvalidPreviousState();
}
```

Recommendation

The team is advised to remove the alternative path that validates against nonce and require that `onchainNonce` exactly equals the currently stored on-chain nonce for the channel asset.

Team Update

The team has acknowledged that this is not a security issue and states:

Intentional design. Allowing validation against the submitted nonce is a deliberate recovery mechanism: it permits a user who has lost off-chain state (or only observes an older on-chain anchor) to attempt a unilateral settlement anchored to a state they can produce, while remaining subject to challenge and penalty if the counterparty (or a watchtower) holds a strictly newer co-signed state.

PDS - Permissionless Dispute Settlement

Criticality	Minor / Informative
Location	libraries/LithiumSettler.sol#L134,233
Status	Acknowledged

Description

The contract restricts callers for unilateral settlement only when no dispute exists, but once a dispute has been created the caller check no longer applies, allowing any third party to invoke unilateral settlement flows. However, unilateral settlement during an active dispute can update dispute-related state, incorporate new payment preimages, and potentially influence which commitment is considered in effect.

```
if (dispute.disputeDeadline == 0 && msg.sender != channelConfig.alice
    && msg.sender != channelConfig.bob) {
    revert InvalidSender();
}
```

Recommendation

The team is advised to enforce caller authorization throughout the entire dispute lifecycle unless a deliberate watchtower model is intended and formally supported. If watchtowers are desired, access should be explicitly controlled.

Team Update

The team has acknowledged that this is not a security issue and states:

Intentional design. The protocol intentionally allows permissionless third-party updates during the dispute window to enable watchtower-style monitoring and recovery services. This model improves liveness and user safety by allowing trusted delegations (or any interested party) to submit co-signed higher-nonce states or full HTLC lists when a participant is offline or has lost state.

PTAI - Potential Transfer Amount Inconsistency

Criticality	Minor / Informative
Location	libraries/LithiumTransfer.sol#L91
Status	Acknowledged

Description

The `transfer()` and `transferFrom()` functions are used to transfer a specified amount of tokens to an address. The fee or tax is an amount that is charged to the sender of a token when tokens are transferred to another address. According to the specification, the transferred amount could potentially be less than the expected amount. This may produce inconsistency between the expected and the actual behavior.

The following example depicts the diversion between the expected and actual amount.

Tax	Amount	Expected	Actual
No Tax	100	100	100
10% Tax	100	100	90

```
function depositAssetFrom(address from, AssetVariant variant, address
asset, uint256 id, uint256 amount) internal {
    if (variant != AssetVariant.Native && asset == address(0)) revert
InvalidAssetAddress();
    if (variant == AssetVariant.ERC20) {
        IERC20(asset).safeTransferFrom(from, address(this), amount);
    }
    // ...
}
```

Recommendation

The team is advised to take into consideration the actual amount that has been transferred instead of the expected.

It is important to note that a transfer tax is not a standard feature of the tokens' specification, and it is not universally implemented by all token contracts. Therefore, the contract could produce the actual amount by calculating the difference between the transfer call.

```
Actual Transferred Amount = Balance After Transfer - Balance  
Before Transfer
```

Team Update

The team has acknowledged that this is not a security issue and states:

This finding is addressed by the protocol's permissionless design model. The security architecture does not rely on on-chain enforcement against non-standard token behavior, but instead on isolation, mutual agreement, and reentrancy protection. No code changes are required; the issue is mitigated by design and documentation.

SSS - Stale State Selection

Criticality	Minor / Informative
Location	libraries/LithiumSettler.sol#L151,250,497
Status	Acknowledged

Description

When a dispute is opened by one party, the non-disputer counterparty has the right to immediately confirm settlement at any time without waiting for the dispute deadline. Combined with the higher-nonce override mechanism, this allows the non-disputer to strategically pick which co-signed state to broadcast in order to maximize their own share.

```
if (dispute.disputeDeadline == 0 || nonce > dispute.nonce) {
    bytes32 storedStateHash = store.assetChannels[assetChannelId];
    _validatePreviousStateAndUpdateDispute(
        dispute,
        storedStateHash,
    );
}

{
    bool disputerIsAlice = dispute.disputerIsAlice;
    if (!(disputerIsAlice ? msg.sender == bob : msg.sender == alice)) {
        if (block.timestamp < dispute.disputeDeadline) revert
        DisputePeriodNotPassed();
    }
}
```

Recommendation

The protocol should encourage disputers to always broadcast the most recent co-signed state they have, since opening a dispute with anything older invites the counterparty to pick a more favorable historical state.

Team Update

The team has acknowledged that this is not a security issue and states:

No smart contract change is needed. Stale-state broadcasting is economically suicidal: the honest party holds cumulative revocations for all superseded states and can seize all channel funds via `penalizeRevokedChannel`. The challenge period serves as a secondary defense when revocations are unavailable. This is intrinsic to all revocation-based payment channel protocols.

UPH - Unsafe Packed Hashing

Criticality	Minor / Informative
Location	libraries/LithiumVerification.sol#L128
Status	Acknowledged

Description

The contract uses `abi.encodePacked(...)` to construct the message that is later hashed and signed. However, `abi.encodePacked` produces a tightly packed, non-self-delimiting byte array that can lead to ambiguous encodings across different input combinations, especially when multiple values are concatenated. Even though the current parameters are mostly fixed-size types, the presence of enums and mixed-size values introduces a risk of unintended collisions at the byte level. As a result, it is possible for distinct sets of parameters to produce the same packed encoding and hash, which undermines the integrity of the signature verification process and can allow signatures to be replayed or misinterpreted in unintended contexts.

```
return abi.encodePacked(
    address(this),
    block.chainid,
    alice,
    bob,
    channelNumber,
    disputePeriod,
    withdrawSetting,
    assetVariant,
    assetAddress,
    tokenNumberId,
    nonce,
    onchainNonce,
    aliceBalance,
    bobBalance,
    prevAliceBalance,
    prevBobBalance,
    hashlockPaymentsMerkleRoot
).toEthSignedMessageHash();
```

Recommendation

The team is advised to avoid `abi.encodePacked` for security-critical hashing and signature schemes. For stronger guarantees and industry best practices—especially given the critical role of signatures in payment channels—the team should consider migrating to an EIP-712 typed structured data approach with a domain separator and explicit type hashes, ensuring robust, unambiguous, and replay-resistant signature verification.

Team Update

The team has acknowledged that this is not a security issue and states:

With the current fields there is no practical risk of collisions or replay: the signed payload contains only fixed-size types (addresses, uint256, enums and a bytes32 Merkle root) and includes strongly-binding context (address(this) and block.chainid) as well as counters (nonce / onchainNonce). Given that the contract is not upgradeable and that signatures are produced and verified by off-chain software, there is no measurable security benefit to migrating to EIP-712. Migrating to EIP-712 for the current protocol would be primarily an ergonomics/standardization change that would add slight complexity and a small gas cost for on-chain verification.

Function Analysis

Contract	Type	Bases		
	Function Name	Visibility	Mutability	Modifiers
Lithium	Implementation	ILithium, ERC721Holder, ERC1155Holder, ReentrancyGuardTransient		
	updateChannel	External	Payable	nonReentrant
	unilateralSettleChannel	External	✓	nonReentrant
	confirmUnilateralSettlement	External	✓	nonReentrant
	penalizeRevokedChannel	External	✓	nonReentrant
	depositToPool	External	Payable	nonReentrant
	withdrawFromPool	External	✓	nonReentrant
	claimAsset	External	✓	nonReentrant
	batchExecute	External	Payable	nonReentrant
	getLiquidity	External		-
	getAssetDispute	External		-
	getAssetChannel	External		-
	getClaimable	External		-
		External	Payable	nonReentrant
	supportsInterface	Public		-
LithiumVerification	Library			
	verifyAssetChannelStateSignatures	Internal		
	verifyAssetChannelStateSignaturesCalldata	Internal		
	_createAssetChannelStateSignableMessage	Private		
	verifyRevocationSignature	Internal		
	verifyRevocationSignatureCalldata	Internal		
	_createRevocationSignableMessage	Private		
	calculateMerkleRootAndVerifyPreimages	Internal		

	calculateMerkleRootAndVerifyPreimagesCalldata	Internal		
	_buildMerkleTree	Private		
LithiumUpdater	Library			
	updateChannel	Internal	✓	
	updateChannelCalldata	Internal	✓	
	_updateAssetChannel	Internal	✓	
LithiumTransfer	Library			
	processBalanceChanges	Internal	✓	
	depositAssetFrom	Internal	✓	
	withdrawAssetTo	Internal	✓	
	claimAsset	Internal	✓	
	payoutOrEscrow	Internal	✓	
	depositToPool	Internal	✓	
	withdrawFromPool	Internal	✓	
LithiumSettler	Library			
	unilateralSettleChannel	Internal	✓	
	unilateralSettleChannelCalldata	Internal	✓	
	_unilateralSettleAssetChannel	Internal	✓	
	_unilateralSettleAssetChannelCalldata	Internal	✓	
	_validatePreviousStateAndUpdateDispute	Private	✓	
	confirmUnilateralSettlement	Internal	✓	
	confirmUnilateralSettlementCalldata	Internal	✓	
	_confirmAssetUnilateralSettlement	Internal	✓	
	_resolveHTLCs	Private		
LithiumRevoker	Library			
	penalizeRevokedChannel	Internal	✓	
	penalizeRevokedChannelCalldata	Internal	✓	
	_revokeAssetChannel	Private	✓	

ILithium	Interface			
	updateChannel	External	Payable	-
	unilateralSettleChannel	External	✓	-
	confirmUnilateralSettlement	External	✓	-
	penalizeRevokedChannel	External	✓	-
	depositToPool	External	Payable	-
	withdrawFromPool	External	✓	-
	batchExecute	External	Payable	-
	getLiquidity	External		-
	getAssetDispute	External		-
	getAssetChannel	External		-
	claimAsset	External	✓	-
	getClaimable	External		-
		External	Payable	-

Inheritance Graph



Summary

Hydranet contract implements a payment channel mechanism. This audit investigates security issues, business logic concerns and potential improvements.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security. Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



A **TAC Security** Company

The Cyberscope team

cyberscope.io